

Freestanding Library: Rewording the Status Quo

Document number: P1641R1

Date: 2019-09-29

Reply-to: Ben Craig <ben dot craig at gmail dot com>

Audience: SG14, Library Evolution Working Group

Change history

R1

- Rebased to N4830
- Dealt with synchronization library additions
- Dealt with `source_location` addition
- Updated feature test macros
- Made new feature test macro name more consistent with other library names

R0

Branching from P0829R4. This "omnibus" paper is still the direction I am aiming for. However, it is too difficult to review. It needs to change with almost every meeting. Therefore, it is getting split up into smaller, more manageable chunks.

Limiting paper to `<version>`, blanket wording, and freestanding facilities already in the working draft.

Introduction

This paper is the first of many smaller papers adding facilities to the C++ freestanding library. This paper in particular will be updating the editorial technique for declaring facilities as freestanding and adjusting feature test macros. This update will make it easier to mark headers as partially freestanding. This wording change will make it easier to mark in-flight proposals as freestanding, without causing a major blocking point in the standardization process.

Motivation and Design

Many existing facilities in the C++ standard library could be used without trouble in freestanding environments. This series of papers will specify the maximal subset of the C++ standard library that does not require an OS or space overhead.

For a more in depth rationale, see P0829.

This paper is going to add blanket wording to make it easier to mark things freestanding, and it will retrofit the existing freestanding facilities. It also includes a non-normative note that should allow implementers to use existing no-exception builds of their libraries as an implementation defined extensions, with a fair number of qualifications. Here's that note, reproduced so that it can be debated more effectively:

[*Note*: Throwing a standard library provided exception is not observably different from `terminate()` if the implementation doesn't unwind the stack during exception handling and the user's program contains no catch blocks. *-end note*]

[Note: Throwing a standard library provided exception is not observably different from terminate() if the implementation doesn't unwind the stack during exception handling and the user's program contains no catch blocks. -end note]

In the associated header synopsis for such declarations and macro definitions, the items shall be followed with a comment that ends with *freestanding*, as in:

```
#define E2BIG see below //freestanding
```

- 2 The *freestanding* implementation has several headers that shall meet the same requirements as for a hosted implementation unless otherwise specified. The synopsis for these headers shall start with a comment that ends with *freestanding*, as in:

```
//freestanding  
namespace std {
```

Individual declarations and macro definitions in such a header shall not be followed with a comment that ends with *freestanding*.

- 3 Deduction guides for class templates that are implemented (partially or fully) in a *freestanding* implementation shall be implemented in a *freestanding* implementation. The containing namespace for each non-namespace declaration that is implemented (partially or fully) in a *freestanding* implementation shall be implemented in a *freestanding* implementation.

Change in [compliance] paragraph 3:

~~The supplied version of the header `<cstdlib>` shall declare at least the functions `abort`, `atexit`, `at_quick_exit`, `exit`, and `quick_exit` ([support.start.term]). The supplied version of the header `<atomic>` shall meet the same requirements as for a hosted implementation except that support for always lock-free integral atomic types ([atomics.lockfree]) is implementation-defined, and whether or not the type aliases `atomic_signed_lock_free` and `atomic_unsigned_lock_free` are defined ([atomics.alias]) is implementation-defined. The other headers listed in this table shall meet the same requirements as for a hosted implementation.~~ **The supplied versions of the header `<version>` shall meet the requirements specified in [support.limits.general]. The other headers listed in this table shall meet the requirements for a *freestanding* implementation, as specified in the respective header synopsis.**

Instructions to the editor:

Please add a *//freestanding* comment to the beginning of the following synopses. These headers are entirely *freestanding*.

- **[cstddef.syn]**
- **[limits.syn]**
- **[climits.syn]**
- **[cfloat.syn]**
- **[cstdint.syn]**
- **[new.syn]**
- **[typeinfo.syn]**
- **[source_location.syn]**
- **[exception.syn]**
- **[initializer.list.syn]**
- **[coroutine.syn]**
- **[cstdarg.syn]**
- **[concepts.syn]**
- **[meta.type.synop]**

- **[bit.syn]**

Change in [cstdlib.syn]:

???

Header <cstdlib> synopsis

[cstdlib.syn]

```
namespace std {
    using size_t = see below; // freestanding
    using div_t = see below;
    using ldiv_t = see below;
    using lldiv_t = see below;
}

#define NULL see below // freestanding
#define EXIT_FAILURE see below
#define EXIT_SUCCESS see below
#define RAND_MAX see below
#define MB_CUR_MAX see below

namespace std {
    // Exposition-only function type aliases
    extern "C" using c-atexit-handler = void(); // exposition only
    extern "C++" using atexit-handler = void(); // exposition only
    extern "C" using c-compare-pred = int(const void*, const void*); // exposition only
    extern "C++" using compare-pred = int(const void*, const void*); // exposition only

    // [support.start.term], start and termination
    [[noreturn]] void abort() noexcept; // freestanding
    int atexit(c-atexit-handler* func) noexcept; // freestanding
    int atexit(atexit-handler* func) noexcept; // freestanding
    int at_quick_exit(c-atexit-handler* func) noexcept; // freestanding
    int at_quick_exit(atexit-handler* func) noexcept; // freestanding
    [[noreturn]] void exit(int status); // freestanding
    [[noreturn]] void _Exit(int status) noexcept;
    [[noreturn]] void quick_exit(int status) noexcept; // freestanding

    char* getenv(const char* name);
    int system(const char* string);

    // [c.malloc], C library memory allocation
    void* aligned_alloc(size_t alignment, size_t size);
    void* calloc(size_t nmemb, size_t size);
    void free(void* ptr);
    void* malloc(size_t size);
    void* realloc(void* ptr, size_t size);

    double atof(const char* nptr);
    int atoi(const char* nptr);
    long int atol(const char* nptr);
    long long int atoll(const char* nptr);
    double strtod(const char* nptr, char** endptr);
    float strttof(const char* nptr, char** endptr);
    long double strtold(const char* nptr, char** endptr);
    long int strtol(const char* nptr, char** endptr, int base);
    long long int strtoll(const char* nptr, char** endptr, int base);
    unsigned long int strtoul(const char* nptr, char** endptr, int base);
```

```

unsigned long long int strtoull(const char* nptr, char** endptr, int base);

// [c.mb.wcs], multibyte / wide string and character conversion functions
int mblen(const char* s, size_t n);
int mbtowc(wchar_t* pwc, const char* s, size_t n);
int wctomb(char* s, wchar_t wchar);
size_t mbstowcs(wchar_t* pwcs, const char* s, size_t n);
size_t wcstombs(char* s, const wchar_t* pwcs, size_t n);

// [alg.c.library], C standard library algorithms
void* bsearch(const void* key, const void* base, size_t nmemb, size_t size,
              c-compare-pred* compar);
void* bsearch(const void* key, const void* base, size_t nmemb, size_t size,
              compare-pred* compar);
void qsort(void* base, size_t nmemb, size_t size, c-compare-pred* compar);
void qsort(void* base, size_t nmemb, size_t size, compare-pred* compar);

// [c.math.rand], low-quality random number generation
int rand();
void srand(unsigned int seed);

// [c.math.abs], absolute values
int abs(int j);
long int abs(long int j);
long long int abs(long long int j);
float abs(float j);
double abs(double j);
long double abs(long double j);

long int labs(long int j);
long long int llabs(long long int j);

div_t div(int numer, int denom);
ldiv_t div(long int numer, long int denom); // see [library.c]
lldiv_t div(long long int numer, long long int denom); // see [library.c]
ldiv_t ldiv(long int numer, long int denom);
lldiv_t lldiv(long long int numer, long long int denom);
}

```

Replace paragraph 3 in [support.limits.general]:

~~3 The macros in Table 36 are defined after inclusion of the header `<version>` or one of the corresponding headers specified in the table.
[Note: Future versions of this International Standard might replace the values of these macros with greater values. -end note]~~

**3 On hosted implementations, the macros in Table ???6?? shall be defined to the corresponding integer literal after inclusion of the header `<version>` or one of the corresponding headers specified in the table. On freestanding implementations, the indicated macros in Table ???6?? are defined after inclusion of the header `<version>` or one of the corresponding headers specified in the table. The presence of the non-indicated macros in Table ???6?? is implementation defined on freestanding implementations.
[Note: Future versions of this International Standard might replace the values of these macros with greater values. -end note]**

Modify Table ??36?? as indicated:

Table ??36??: Standard library feature-test macros

Macro name	Value	Freestanding	Header(s)
__cpp_lib_addressof_constexpr	201603L		<memory>
__cpp_lib_allocator_traits_is_always_equal	201411L		<memory> <scoped_allocator> <string> <deque> <forward_list> <list> <vector> <map> <set> <unordered_map> <unordered_set>
__cpp_lib_any	201606L		<any>
__cpp_lib_apply	201603L		<tuple>
__cpp_lib_array_constexpr	201603L		<iterator> <array>
__cpp_lib_as_const	201510L		<utility>
__cpp_lib_atomic_flag_test	201907L	+	<atomic>
__cpp_lib_atomic_is_always_lock_free	201603L	+	<atomic>
__cpp_lib_atomic_lock_free_type_aliases	201907L		<atomic>
__cpp_lib_atomic_ref	201806L	+	<atomic>
__cpp_lib_atomic_wait	201907L	+	<atomic>
__cpp_lib_barrier	201907L		<barrier>
__cpp_lib_bit_cast	201806L	+	<bit>
__cpp_lib_bind_front	201907L		<functional>
__cpp_lib_bitops	201907L	+	<bit>
__cpp_lib_bool_constant	201505L	+	<type_traits>
__cpp_lib_bounded_array_traits	201902L	+	<type_traits>
__cpp_lib_boyer_moore_searcher	201603L		<functional>
__cpp_lib_byte	201603L	+	<cstdint>
__cpp_lib_char8_t	201907L	+	<atomic> <filesystem> <istream> <limits> <locale> <ostream> <string> <string_view>
__cpp_lib_chrono	201907L		<chrono>
__cpp_lib_chrono_udls	201304L		<chrono>
__cpp_lib_clamp	201603L		<algorithm>
__cpp_lib_complex_udls	201309L		<complex>
__cpp_lib_concepts	201806L	+	<concepts>
__cpp_lib_constexpr	201811L		any C++ library header from Table 19 or any C++ header for C library facilities from Table 20
__cpp_lib_constexpr_dynamic_alloc	201907L		<memory>
__cpp_lib_constexpr_invoke	201907L		<functional>
__cpp_lib_constexpr_string	201907L		<string>

Macro name	Value	Freestanding	Header(s)
__cpp_lib_constexpr_swap_algorithms	201806L		<algorithm>
__cpp_lib_constexpr_vector	201907L		<vector>
__cpp_lib_destroying_delete	201806L		<new>
__cpp_lib_enable_shared_from_this	201603L		<memory>
__cpp_lib_endian	201907L	+	<bit>
__cpp_lib_erase_if	201811L		<string> <deque> <forward_list> <list> <vector> <map> <set> <unordered_map> <unordered_set>
__cpp_lib_exchange_function	201304L		<utility>
__cpp_lib_execution	201902L		<execution>
__cpp_lib_filesystem	201703L		<filesystem>
__cpp_lib_format	201907L		<format>
__cpp_lib_freestanding	201906L	+	
__cpp_lib_gcd_lcm	201606L		<numeric>
__cpp_lib_generic_associative_lookup	201304L		<map> <set>
__cpp_lib_generic_unordered_lookup	201811L		<unordered_map> <unordered_set>
__cpp_lib_hardware_interference_size	201703L	+	<new>
__cpp_lib_has_unique_object_representations	201606L	+	<type_traits>
__cpp_lib_hypot	201603L		<cmath>
__cpp_lib_incomplete_container_elements	201505L		<forward_list> <list> <vector>
__cpp_lib_integer_sequence	201304L		<utility>
__cpp_lib_integral_constant_callable	201304L	+	<type_traits>
__cpp_lib_interpolate	201902L		<cmath> <numeric>
__cpp_lib_invoke	201411L		<functional>
__cpp_lib_is_aggregate	201703L	+	<type_traits>
__cpp_lib_is_constant_evaluated	201811L	+	<type_traits>
__cpp_lib_is_final	201402L	+	<type_traits>
__cpp_lib_is_invocable	201703L	+	<type_traits>
__cpp_lib_is_layout_compatible	201907L	+	<type_traits>
__cpp_lib_is_null_pointer	201309L	+	<type_traits>
__cpp_lib_is_pointer_interconvertible	201907L	+	<type_traits>
__cpp_lib_is_swappable	201603L	+	<type_traits>
__cpp_lib_jthread	201907L		<stop_token> <thread>
__cpp_lib_latch	201907L		<latch>
__cpp_lib_launder	201606L	+	<new>
__cpp_lib_list_remove_return_type	201806L		<forward_list> <list>
__cpp_lib_logical_traits	201510L	+	<type_traits>
__cpp_lib_make_from_tuple	201606L		<tuple>
__cpp_lib_make_reverse_iterator	201402L		<iterator>
__cpp_lib_make_unique	201304L		<memory>

Macro name	Value	Freestanding	Header(s)
__cpp_lib_map_try_emplace	201411L		<map>
__cpp_lib_math_constants	201907L		<numbers>
__cpp_lib_math_special_functions	201603L		<cmath>
__cpp_lib_memory_resource	201603L		<memory_resource>
__cpp_lib_node_extract	201606L		<map> <set> <unordered_map> <unordered_set>
__cpp_lib_nonmember_container_access	201411L		<iterator> <array> <deque> <forward_list> <list> <map> <regex> <set> <string> <unordered_map> <unordered_set> <vector>
__cpp_lib_not_fn	201603L		<functional>
__cpp_lib_null_iterators	201304L		<iterator>
__cpp_lib_optional	201606L		<optional>
__cpp_lib_parallel_algorithm	201603L		<algorithm> <numeric>
__cpp_lib_quoted_string_io	201304L		<iomanip>
__cpp_lib_ranges	201811L		<algorithm> <functional> <iterator> <memory> <ranges>
__cpp_lib_raw_memory_algorithms	201606L		<memory>
__cpp_lib_result_of_sfinae	201210L	+	<functional> <type_traits>
__cpp_lib_robust_nonmodifying_seq_ops	201304L		<algorithm>
__cpp_lib_sample	201603L		<algorithm>
__cpp_lib_scoped_lock	201703L		<mutex>
__cpp_lib_semaphore	201907L		<semaphore>
__cpp_lib_shared_mutex	201505L		<shared_mutex>
__cpp_lib_shared_ptr_arrays	201611L		<memory>
__cpp_lib_shared_ptr_weak_type	201606L		<memory>
__cpp_lib_shared_timed_mutex	201402L		<shared_mutex>
__cpp_lib_source_location	201907L	+	<source_location>
__cpp_lib_spaceship	201907L		<compare>
__cpp_lib_string_udls	201304L		<string>
__cpp_lib_string_view	201606L		<string> <string_view>
__cpp_lib_three_way_comparison	201711L		<compare>
__cpp_lib_to_array	201907L		<array>
__cpp_lib_to_chars	201611L		<charconv>
__cpp_lib_transformation_trait_aliases	201304L	+	<type_traits>
__cpp_lib_transparent_operators	201510L		<memory> <functional>
__cpp_lib_tuple_element_t	201402L		<tuple>

Macro name	Value	Freestanding	Header(s)
__cpp_lib_tuples_by_type	201304L		<utility> <tuple>
__cpp_lib_type_trait_variable_templates	201510L	+	<type_traits>
__cpp_lib_uncaught_exceptions	201411L	+	<exception>
__cpp_lib_unordered_map_try_emplace	201411L		<unordered_map>
__cpp_lib_variant	201606L		<variant>
__cpp_lib_void_t	201411L	+	<type_traits>

Instructions to the editor:

Please add a `// freestanding` comment to all entities in [atomics.syn] other than `atomic_signed_lock_free` and `atomic_unsigned_lock_free`

Change in [atomics.lockfree] paragraph 2:

On hosted implementations ([compliance]), ~~a~~At least one signed integral specialization of the atomic template, along with the specialization for the corresponding unsigned type ([basic.fundamental]), is always lock-free.

Note

: This requirement is optional in freestanding implementations ([compliance]). — end note

Acknowledgements

Thanks to Brandon Streiff, Joshua Cannon, Phil Hindman, and Irwan Djajadi for reviewing P0829.

Thanks to Odin Holmes for providing feedback and helping publicize P0829.

Thanks to Paul Bendixen for providing feedback while prototyping P0829.

Similar work was done in the C++11 timeframe by Lawrence Crowl and Alberto Ganesh Barbati in [N3256](#).